

MVC Practical

Step 1 Project Structure

Create Folder DemoMVC

Open Demo folder with vscode

Open terminal

Create Folder MVC

```
md MVC
```

```
cd MVC
```

```
dotnet new mvc
```

Create Folder Repositories

```
md Repositories
```

```
cd Repositories
```

```
dotnet new classlib
```

Add Package in Repositories

```
dotnet add package npgsql
```

```
dotnet add package Microsoft.AspNetCore.Http
```

Create Folders in Repositories

```
md Implementations
```

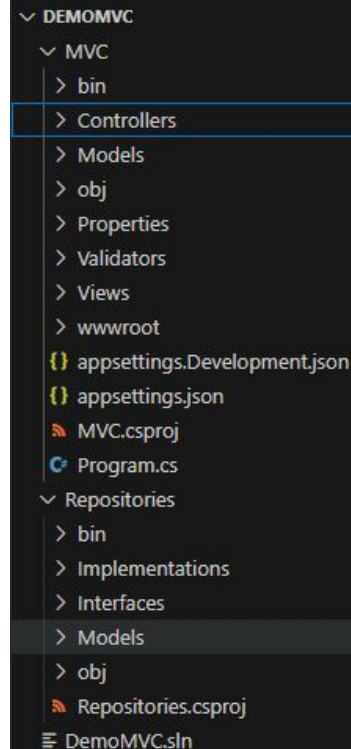
```
md Interfaces
```

```
md Models
```

Add Repository Reference to your project

```
cd ../MVC
```

```
dotnet add reference ../Repositories
```



Step 2 Postgre

Create table t_User in postgre

```
CREATE TABLE t_user (  
    c_userid integer NOT NULL GENERATED ALWAYS AS IDENTITY  
    ( INCREMENT 1 START 1 MINVALUE 1 MAXVALUE 2147483647 CACHE 1 ),  
    c_username character varying(100) COLLATE pg_catalog."default" NOT NULL,  
    c_email character varying(100) COLLATE pg_catalog."default" NOT NULL,  
    c_password character varying(100) COLLATE pg_catalog."default" NOT NULL,  
    c_address character varying(500) COLLATE pg_catalog."default",  
    c_mobile character varying(50) COLLATE pg_catalog."default",  
    c_gender character varying(10) COLLATE pg_catalog."default",  
    c_image character varying(4000) COLLATE pg_catalog."default",  
    CONSTRAINT "tblUser_pkey" PRIMARY KEY (c_userid)  
)
```

Step 2 Postgre

Create table t_contact in postgre

```
CREATE TABLE IF NOT EXISTS public.t_contact
(
    c_contactid integer NOT NULL GENERATED ALWAYS AS IDENTITY
    ( INCREMENT 1 START 1 MINVALUE 1 MAXVALUE 2147483647 CACHE 1 ),
    c_userid integer NOT NULL,
    c_contactname character varying(100) NOT NULL,
    c_email character varying(100) NOT NULL,
    c_address character varying(500) ,
    c_mobile character varying(50) ,
    c_group character varying(50) ,
    c_image character varying(4000) ,
    c_status character varying(20) ,
    CONSTRAINT t_contact_pkey PRIMARY KEY (c_contactid)
)
```

Step 3 Repositories/Models

```
dotnet new class -n t_User
```

```
using System.ComponentModel.DataAnnotations;  
using System.ComponentModel.DataAnnotations.Schema;  
using Microsoft.AspNetCore.Http;  
namespace Repositories;  
public class t_User  
{  
    [Key]  
    [DatabaseGenerated(DatabaseGeneratedOption.Identity)]  
    public int c_UserId { get; set; }  
  
    [StringLength(100)]  
    [Required(ErrorMessage = "Username is required.")]  
    public string c_UserName { get; set; }  
}
```

Step 3 Repositories/Models

```
[StringLength(100)]  
[Required(ErrorMessage = "Email is required.")]  
[EmailAddress(ErrorMessage = "Invalid email format.")]  
public string c_Email { get; set; }
```

```
[StringLength(100)]  
[Required(ErrorMessage = "Password is required.")]  
[MinLength(6, ErrorMessage = "Password must be at least 6 characters  
long.")]  
public string c_Password { get; set; }
```

```
[StringLength(100)]  
[Required(ErrorMessage = "Confirm Password is required.")]  
[Compare("c_Password", ErrorMessage = "Passwords do not match.")]  
public string c_ConfirmPassword { get; set; }
```

Step 3 Repositories/Models

```
[StringLength(500)]  
public string? c_Address { get; set; }
```

```
[StringLength(50)]  
public string? c_Mobile { get; set; }
```

```
[StringLength(10)]  
public string? c_Gender { get; set; }
```

```
[StringLength(4000)]  
public string? c_Image { get; set; }
```

```
public IFormFile? ProfilePicture { get; set; }
```

```
}
```

Step 4 Repositories/Models

```
dotnet new class -n vm_Login

using System.ComponentModel.DataAnnotations;
using Microsoft.AspNetCore.Http;
namespace Repositories;
public class vm_Login
{
    [StringLength(100)]
    [Required(ErrorMessage = "Email is required.")]
    [EmailAddress(ErrorMessage = "Invalid email format.")]
    public string c_Email { get; set; }

    [StringLength(100)]
    [Required(ErrorMessage = "Password is required.")]
    public string c_Password { get; set; }
}
```

Step 5 Repositories/Interfaces

```
dotnet new interface -n IUserInterface;  
namespace Repositories;
```

```
public interface IUserInterface  
{  
    Task<int> Register(t_User user);  
  
    Task<t_User> Login(vm_Login user);  
}
```

Step 6 Repositories/Implementations

```
public class UserRepository : IUserInterface
{
    public async Task<int> Register(t_User data)
    {
    }
    public async Task<t_User> Login(vm_Login user)
    {
    }
}
```

Step 7 MVC/appsettings.json

```
"ConnectionStrings": {  
  "pgconn": "Server=localhost;Port=5432;Database=stud2025;User Id=postgres;Password=Admin@123;"  
},
```

MVC/Program.cs

```
builder.Services.AddSingleton<IUserInterface, UserRepository>();  
builder.Services.AddSingleton<NpgsqlConnection>((Conn) =>  
{  
    var connectionString = Conn.GetRequiredService<IConfiguration>().GetConnectionString("pgconn");  
    return new NpgsqlConnection(connectionString);  
});  
builder.Services.AddSession(options =>  
{  
    options.IdleTimeout = TimeSpan.FromMinutes(30); // Set session timeout  
    options.Cookie.HttpOnly = true;  
    options.Cookie.IsEssential = true;  
});  
...  
var app = builder.Build();  
...  
app.UseSession();  
...
```

Step 8 Repositories/Implementations

```
public class UserRepository : IUserInterface
{
    private readonly NpgsqlConnection _conn;
    public UserRepository(NpgsqlConnection connection)
    {
        _conn = connection;
    }
    public async Task<int> Register(t_User data)
    {
        int status = 0;
        try
        {
            await _conn.CloseAsync();
            NpgsqlCommand comcheck = new NpgsqlCommand(@"SELECT * FROM t_User
                                                         WHERE c_email = @c_email ;", _conn);
            comcheck.Parameters.AddWithValue("@c_email", data.c_Email);
```

Step 8 Repositories/Implementations

```
        await _conn.OpenAsync();
using (NpgsqlDataReader datadr = await comcheck.ExecuteReaderAsync())
{
    if (datadr.HasRows)
    {
        await _conn.CloseAsync();
        return 0;
    }
    else
    {
        await _conn.CloseAsync();
        NpgsqlCommand com = new NpgsqlCommand(@"INSERT INTO t_User
            (c_username,c_email,c_password,c_address,
            c_gender,c_mobile,c_image) VALUES (@c_userName,
            @c_email, @c_password,@c_address,@c_gender,
            @c_mobile,@c_image)", _conn);
```

Step 8 Repositories/Implementations

```
com.Parameters.AddWithValue("@c_userName", data.c_UserName);
com.Parameters.AddWithValue("@c_email", data.c_Email);
com.Parameters.AddWithValue("@c_password", data.c_Password);
com.Parameters.AddWithValue("@c_address", data.c_Address);
com.Parameters.AddWithValue("@c_gender", data.c_Gender);
com.Parameters.AddWithValue("@c_mobile", data.c_Mobile);
com.Parameters.AddWithValue("@c_image", data.c_Image);
await _conn.OpenAsync();
await com.ExecuteNonQueryAsync();
await _conn.CloseAsync();
return 1; //returning 1 is user has Successfully Registered
    }
}
}
```

Step 8 Repositories/Implementations

```
        catch (Exception e)
        {
            await _conn.CloseAsync();
            Console.WriteLine("Register Failed , Error :- " + e.Message);
            return -1; //-1 if There is error during registration
        }
    }

    public async Task<t_User> Login(vm_Login user)
    {
        t_User UserData = new t_User();
        var qry = "SELECT * FROM t_user WHERE c_email=@c_email AND  
                c_password=@c_password;";
```

Step 8 Repositories/Implementations

```
try
{
    using (NpgsqlCommand cmd = new NpgsqlCommand(qry, _conn))
    {
        cmd.Parameters.AddWithValue("@c_email", user.c_Email);
        cmd.Parameters.AddWithValue("@c_password", user.c_Password);
        await _conn.OpenAsync();
        var reader = await cmd.ExecuteReaderAsync();
        if (reader.Read())
        {
            UserData.c_UserId = (int)reader["c_userid"];
            UserData.c_UserName = (string)reader["c_username"];
            UserData.c_Email = (string)reader["c_email"];
            UserData.c_Gender = (string)reader["c_gender"];
            UserData.c_Mobile = (string)reader["c_mobile"];
            UserData.c_Address = (string)reader["c_address"];
            UserData.c_Image = (string)reader["c_image"];
        }
    }
}
```

Step 8 Repositories/Implementations

```
        }  
    }  
}  
catch (Exception e)  
{  
    Console.WriteLine("----->Login Error : " + e.Message);  
}  
finally  
{  
    await _conn.CloseAsync();  
}  
  
return UserData;  
}  
}
```

Step 9 MVC/HomeController.cs

```
private readonly IUserInterface _userRepo;

public HomeController(ILogger<HomeController> logger, IUserInterface userRepo)
{
    _logger = logger;
    _userRepo = userRepo;
}

public IActionResult Index()
{
    return View();
}

public IActionResult Login()
{
    return View();
}
```

Step 9 MVC/HomeController.cs

```
[HttpPost]
public async Task<IActionResult> Login(vm_Login login)
{
    t_User UserData = await _userRepo.Login(login);
    if (ModelState.IsValid)
    {
        if (UserData.c_UserId != 0)
        {
            HttpContext.Session.SetInt32("UserId", UserData.c_UserId);
            HttpContext.Session.SetString("UserName", UserData.c_UserName);
            return RedirectToAction("Index", "ContactSingle");
        }
        else
        {
            ViewData["message"] = "Invalid Username and Password";
        }
    }
}
```

Step 9 MVC/HomeController.cs

```
        return View(login);
    }
    public IActionResult Register()
    {
        return View();
    }
    [HttpPost]
    public async Task<IActionResult> Register(t_User user)
    {
        if (ModelState.IsValid)
        {
            if (user.ProfilePicture != null && user.ProfilePicture.Length > 0)
            {
                // Save the uploaded file
                var fileName = user.c_Email + Path.GetExtension(user.ProfilePicture.FileName);
                var filePath = Path.Combine("../MVC/wwwroot/profile_images", fileName);
```

Step 9 MVC/HomeController.cs

```
Directory.CreateDirectory(Path.Combine("../MVC/wwwroot/profile_images"));
    user.c_Image = fileName;
    using (var stream = new FileStream(filePath, FileMode.Create))
    {
        user.ProfilePicture.CopyTo(stream);
    }
}
Console.WriteLine("user.c_fname: " + user.c_UserName);
var status = await _userRepo.Register(user);
if (status == 1)
{
    ViewData["message"] = "User Registered";
    return RedirectToAction("Login");
}
```

Step 9 MVC/HomeController.cs

```
        else if (status == 0)
        {
            ViewData["message"] = "User Already Registred";
        }
        else
        {
            ViewData["message"] = "There was some error while Registration";
        }
    }
    return View();
}
```

Step 10 MVC/Home/Register.cshtml

```
@using Repositories
@model t_User
@{
    ViewData["Title"] = "Register";
}
<div class="container mt-5" style="border: 1px solid #ccc; width: 70%;padding:20px">
    <h3>Register</h3>
    <form enctype="multipart/form-data" asp-controller="Home" asp-action="Register" method="post" >
        <div class="mt-3">
            <label class="control-label mt-1" for="name">Name:</label>
            <div class="mt-1">
                <input type="text" class="form-control" asp-for="c_UserName" id="name" placeholder="Enter name">
            </div>
            <span class="text-danger" asp-validation-for="c_UserName"></span>
        </div>
        <div class="mt-3">
            <label class="control-label mt-1" for="email">Email:</label>
            <div class="mt-1">
                <input type="email" class="form-control" asp-for="c_Email" id="email" placeholder="Enter email">
            </div>
            <span class="text-danger" asp-validation-for="c_Email"></span>
        </div>
    </form>
</div>
```

Step 10 MVC/Home/Register.cshtml

```
<div class="mt-3">
  <label class="control-label mt-1" for="pwd">Password:</label>
  <div class="mt-1">
    <input type="password" class="form-control" asp-for="c_Password" id="pwd" placeholder="Enter password">
  </div>
  <span class="text-danger" asp-validation-for="c_Password"></span>
</div>
<div class="mt-3">
  <label class="control-label mt-1" for="pwd">Confirm Password:</label>
  <div class="mt-1">
    <input type="password" class="form-control" asp-for="c_ConfirmPassword" id="conpwd" placeholder="Enter password">
  </div>
  <span class="text-danger" asp-validation-for="c_ConfirmPassword"></span>
</div>
<div class="mt-3">
  <label class="control-label mt-1" for="address">Address:</label>
  <div class="mt-1">
    <input type="text" class="form-control" asp-for="c_Address" id="address" placeholder="Enter address">
  </div>
</div>
```

Step 10 MVC/Home/Register.cshtml

```
<div class="mt-3">
  <label class="control-label mt-1" for="mobile">Mobile:</label>
  <div class="mt-1">
<input type="text" class="form-control" asp-for="c_Mobile" id="mobile" placeholder="Enter mobile">
  </div>
</div>
<div class="mt-3">
  <label class="control-label mt-1" for="gender" Gender:</label>
  <div class="mt-1">
    <input type="radio" id="gender1" asp-for="c_Gender" value="Male"> Male
    <input type="radio" id="gender2" asp-for="c_Gender" value="Female"> Female
    <input type="radio" id="gender3" asp-for="c_Gender" value="Other"> Other
  </div>
</div>
<div class="mt-3">
  <label class="control-label mt-1" for="image">Image:</label>
  <div class="mt-1">
    <input type="file" class="form-control" asp-for="ProfilePicture" id="Image">
  </div>
  <span class="text-danger" asp-validation-for="ProfilePicture"></span>
</div>
```

Step 10 MVC/Home/Register.cshtml

```
<div asp-validation-summary="All" class="text-danger"></div>

<div id="responseMessage" class="mt-3">
    @ViewData["Message"]
</div>

<div class="mt-3">
    <div class="mt-1">
        <button type="submit" class="btn btn-primary" id="register">Register</button>
    </div>
</div>
</form>
</div>
@section Scripts {
    <partial name="_ValidationScriptsPartial" />
}
```

Step 11 MVC/Home/Login.cshtml

```
@using Repositories
@model vm_Login
@{
    ViewData["Title"] = "Home Page";
}
<div class="container mt-5" style="border: 1px solid #ccc; width: 70%;padding:20px">
    <h3>Login</h3>
    <br />
    <form asp-controller="Home" asp-action="Login" method="post">
        <div class="mt-3">
            <label class="control-label mt-1" for="email">Email:</label>
            <div class="mt-1">
                <input type="email" class="form-control" asp-for="c_Email" id="email" placeholder="Enter email">
            </div>
            <span class="text-danger" asp-validation-for="c_Email"></span>
        </div>
        <div class="mt-3">
            <label class="control-label mt-1" for="pwd">Password:</label>
            <div class="mt-1">
                <input type="password" class="form-control" asp-for="c_Password" id="pwd" placeholder="Enter password">
            </div>
            <span class="text-danger" asp-validation-for="c_Password"></span>
        </div>
    </form>
</div>
```

Step 11 MVC/Home/Login.cshtml

```
<div id="responseMessage" class="mt-3">
  <div class="alert @(ViewData["message"] != null ? "alert-danger" : "" )">
    @ViewData["Message"]
  </div>
</div>
<div class="mt-3">
  <div class="mt-1">
    <button type="submit" id="login" class="btn btn-primary">Login</button>
  </div>
</div>
</form>
</div>
```

Step 12 MVC/Shared/_Layout.cshtml

```
...  
    <li class="nav-item">  
        <a class="nav-link text-dark" asp-area="" asp-controller="Home" asp-action="Login">Login</a>  
    </li>  
    <li class="nav-item">  
        <a class="nav-link text-dark" asp-area="" asp-controller="Home" asp-action="Register">Register</a>  
    </li>  
...
```

Step 13 Repositories/Models

```
public class t_Contact
{
    [Key]
    [DatabaseGenerated(DatabaseGeneratedOption.Identity)]
    public int c_ContactId { get; set; }

    public int c_UserId { get; set; }

    [StringLength(100)]
    public string c_ContactName { get; set; }

    [Required]
    [StringLength(100)]
    public string c_Email { get; set; }

    [Required]
    [StringLength(50)]
    public string c_Group { get; set; }
```

Step 13 Repositories/Models

```
[StringLength(500)]  
public string? c_Address { get; set; }
```

```
[StringLength(50)]  
public string? c_Mobile { get; set; }
```

```
[StringLength(4000)]  
public string? c_Image { get; set; }
```

```
[StringLength(20)]  
public string? c_Status { get; set; }
```

```
public IFormFile? ContactPicture { get; set; }
```

```
public static implicit operator t_Contact(List<t_Contact> v)  
{  
    throw new NotImplementedException();  
}
```

```
}
```

Step 13 Repositories/Interfaces

```
public interface IUserInterface
{
    ...
    Task<t_User> GetUser(int userid);
}
```

Step 13 Repositories/Implementations

```
public class UserRepository : IUserInterface
{
    ...
    public async Task<t_User> GetUser(int userid)
    {
        t_User UserData = new t_User();
        var qry = "SELECT * FROM t_user WHERE c_userid=@c_userid;";
        try
        {
            using (NpgsqlCommand cmd = new NpgsqlCommand(qry, _conn))
            {
                cmd.Parameters.AddWithValue("@c_userid", userid);
                await _conn.OpenAsync();
                var reader = await cmd.ExecuteReaderAsync();
                if (reader.Read())
                {
                    UserData.c_UserId = (int)reader["c_userid"];
                    UserData.c_UserName = (string)reader["c_username"];
                    UserData.c_Email = (string)reader["c_email"];
                }
            }
        }
    }
}
```

Step 13 Repositories/Implementations

```
        UserData.c_Gender = (string)reader["c_gender"];
        UserData.c_Mobile = (string)reader["c_mobile"];
        UserData.c_Address = (string)reader["c_address"];
        UserData.c_Image = (string)reader["c_image"];
    }
}
}
catch (Exception e)
{
    Console.WriteLine("----->Login Error : " + e.Message);
}
finally
{
    await _conn.CloseAsync();
}
return UserData;
}
}
```

Step 13 Repositories/Interfaces

```
public interface IContactInterface
{
    Task<List<t_Contact>> GetAll();
    Task<List<t_Contact>> GetAllByUser(string userid);
    Task<t_Contact> GetOne(string contactid);
    Task<int> Add(t_Contact contactData);
    Task<int> Update(t_Contact contactData);
    Task<int> Delete(string contactid);
}
```

Step 13 Repositories/Interfaces

```
public interface IContactInterface
{
    Task<List<t_Contact>> GetAll();
    Task<List<t_Contact>> GetAllByUser(string userid);
    Task<t_Contact> GetOne(string contactid);
    Task<int> Add(t_Contact contactData);
    Task<int> Update(t_Contact contactData);
    Task<int> Delete(string contactid);
}
```

Step 13 Repositories/Implementations

```
public class ContactRepository : IContactInterface
{
    private readonly NpgsqlConnection _conn;
    public ContactRepository(NpgsqlConnection connection)
    {
        _conn = connection;
    }
    public async Task<int> Add(t_Contact data)
    {
        try {
            NpgsqlCommand cm = new NpgsqlCommand(@"INSERT INTO t_contact
                (c_userid,c_contactname,c_email,c_mobile, c_address,c_image,c_status,c_group) VALUES
                (@c_userid,@c_contactname,@c_email,@c_mobile,@c_address,@c_image,@c_status,@c_group)", _conn);
            cm.Parameters.AddWithValue("@c_userid", data.c_UserId);
            cm.Parameters.AddWithValue("@c_contactname", data.c_ContactName);
            cm.Parameters.AddWithValue("@c_email", data.c_Email);
            cm.Parameters.AddWithValue("@c_mobile", data.c_Mobile);
            cm.Parameters.AddWithValue("@c_address", data.c_Address);
            cm.Parameters.AddWithValue("@c_image",data.c_Image==null ? DBNull.Value : data.c_Image);
            cm.Parameters.AddWithValue("@c_status", data.c_Status);
            cm.Parameters.AddWithValue("@c_group", data.c_Group);
```

Step 13 Repositories/Implementations

```
        _conn.Close();
        _conn.Open();
        cm.ExecuteNonQuery();
        _conn.Close();
        return 1;
    }
    catch (Exception ex)
    {
        return 0;
    }
}

public async Task<int> Delete(string contactid)
{
    try
    {
        NpgsqlCommand cm = new NpgsqlCommand(@"DELETE FROM t_contact
                                                WHERE c_contactid=@c_contactid", _conn);
        cm.Parameters.AddWithValue("@c_contactid", int.Parse(contactid));
        _conn.Close();
        _conn.Open();
        cm.ExecuteNonQuery();
        _conn.Close();
    }
}
```

Step 13 Repositories/Implementations

```
        return 1;
    }
    catch (Exception ex)
    {
        return 0;
    }
}

public async Task<List<t_Contact>> GetAll()
{
    DataTable dt = new DataTable();
    NpgsqlCommand cm = new NpgsqlCommand("select * from t_contact", _conn);
    _conn.Close();
    _conn.Open();
    NpgsqlDataReader datar = cm.ExecuteReader();
    if (datar.HasRows)
    {
        dt.Load(datar);
    }
    List<t_Contact> contactList = new List<t_Contact>();
}
```

Step 13 Repositories/Implementations

```
contactList = (from DataRow dr in dt.Rows
                select new t_Contact()
                {
                    c_ContactId = Convert.ToInt32(dr["c_contactid"]),
                    c_UserId = int.Parse(dr["c_userid"].ToString()),
                    c_ContactName = dr["c_contactname"].ToString(),
                    c_Email = dr["c_email"].ToString(),
                    c_Mobile = dr["c_mobile"].ToString(),
                    c_Address = dr["c_address"].ToString(),
                    c_Image = dr["c_image"].ToString(),
                    c_Group = dr["c_group"].ToString(),
                    c_Status = dr["c_status"].ToString()

                }).ToList();
_conn.Close();
return contactList;
}
```

Step 13 Repositories/Implementations

```
public async Task<List<t_Contact>> GetAllByUser(string userid)
{
    DataTable dt = new DataTable();
    NpgsqlCommand cm = new NpgsqlCommand("select * from t_contact", _conn);
    _conn.Close();
    _conn.Open();
    NpgsqlDataReader datar = cm.ExecuteReader();
    if (datar.HasRows)
    {
        dt.Load(datar);
    }
    List<t_Contact> contactList = new List<t_Contact>();
    contactList = (from DataRow dr in dt.Rows
        where dr["c_userid"].ToString() == userid
        select new t_Contact()
        {
            c_ContactId = Convert.ToInt32(dr["c_contactid"]),
            c_UserId = int.Parse(dr["c_userid"].ToString()),
            c_ContactName = dr["c_contactname"].ToString(),
            c_Email = dr["c_email"].ToString(),
        }
    );
}
```

Step 13 Repositories/Implementations

```
        c_Mobile = dr["c_mobile"].ToString(),
        c_Address = dr["c_address"].ToString(),
        c_Image = dr["c_image"].ToString(),
        c_Group = dr["c_group"].ToString(),
        c_Status = dr["c_status"].ToString()

    })).ToList();
    _conn.Close();
    return contactList;
}

public async Task<t_Contact> GetOne(string contactid)
{
    DataTable dt = new DataTable();
    NpgsqlCommand cm = new NpgsqlCommand("select * from t_contact WHERE c_contactid=@c_contactid",
    _conn);
    cm.Parameters.AddWithValue("@c_contactid", int.Parse( contactid));
    _conn.Close();
    _conn.Open();
    NpgsqlDataReader datar = cm.ExecuteReader();
```

Step 13 Repositories/Implementations

```
t_Contact contact = null;
if (datar.Read())
{
    contact = new t_Contact()
    {
        c_ContactId = Convert.ToInt32(datar["c_contactid"]),
        c_UserId = int.Parse(datar["c_userid"].ToString()),
        c_ContactName = datar["c_contactname"].ToString(),
        c_Email = datar["c_email"].ToString(),
        c_Mobile = datar["c_mobile"].ToString(),
        c_Address = datar["c_address"].ToString(),
        c_Image = datar["c_image"].ToString(),
        c_Group = datar["c_group"].ToString(),
        c_Status = datar["c_status"].ToString()
    };
}
_conn.Close();
return contact;
}
```

Step 13 Repositories/Implementations

```
public async Task<int> Update(t_Contact data)
{
    try
    {
        NpgsqlCommand cm = new NpgsqlCommand(@"UPDATE t_contact set
                                                c_userid=@c_userid,
                                                c_contactname=@c_contactname,
                                                c_email=@c_email,
                                                c_mobile=@c_mobile,
                                                c_address=@c_address,
                                                c_Image=@c_image,
                                                c_Status=@c_status,
                                                c_group=@c_group
                                                WHERE c_contactid=@c_contactid", _conn);
        cm.Parameters.AddWithValue("@c_userid", data.c_UserId);
        cm.Parameters.AddWithValue("@c_contactname", data.c_ContactName);
        cm.Parameters.AddWithValue("@c_email", data.c_Email);
        cm.Parameters.AddWithValue("@c_mobile", data.c_Mobile);
        cm.Parameters.AddWithValue("@c_address", data.c_Address);
        cm.Parameters.AddWithValue("@c_image", data.c_Image==null ? DBNull.Value : data.c_Image);
```

Step 13 Repositories/Implementations

```
        cm.Parameters.AddWithValue("@c_status", data.c_Status);
        cm.Parameters.AddWithValue("@c_group", data.c_Group);
        cm.Parameters.AddWithValue("@c_contactid", data.c_ContactId);
        _conn.Close();
        _conn.Open();
        cm.ExecuteNonQuery();
        _conn.Close();
        return 1;
    }
    catch (Exception ex)
    {
        return 0;
    }
}
```

Step 14 Steps

Add new Controller

```
cd Controllers
dotnet new mvcontroller -n ContactController
```

Program.cs

```
. . . .Services.AddSingleton<IUserInterface, UserRepository>();
builder.Services.AddSingleton<IContactInterface, ContactRepository>();
. . . ..Services.AddSingleton<NpgsqlConnection>((UserRepository) =>
{
    var connectionString =
        UserRepository.GetRequiredService<IConfiguration>().GetConnectionString("pgconn");
    return new NpgsqlConnection(connectionString);
});
```

Step 15 Controllers/ContactController.cs

```
public class ContactController : Controller
{
    private readonly IContactInterface _contact;
    private readonly IUserInterface _userRepo;

    public ContactController(IContactInterface contact, IUserInterface userRepo)
    {
        _contact = contact;
        _userRepo = userRepo;
    }

    // GET: ContactController

    public async Task<ActionResult> Index()
    {
        if (HttpContext.Session.GetInt32("UserId") != null)
        {
            t_User UserData = await _userRepo.GetUser((int)HttpContext.Session.GetInt32("UserId"));
            return View(UserData);
        }
    }
}
```

Step 15 Controllers/ContactController.cs

```
        else
        {
            return RedirectToAction("Index", "Home");
        }
    }

    public async Task<ActionResult> Logout()
    {
        HttpContext.Session.Clear();
        if (HttpContext.Session.GetInt32("UserId") != null)
        {
            return RedirectToAction("IndexAsync", "Contact");
        }
        else
        {
            return RedirectToAction("Index", "Home");
        }
    }
}
```

Step 15 Controllers/ContactController.cs

```
public async Task<ActionResult> List()
{
    if (HttpContext.Session.GetInt32("UserId") != null)
    {
        List<t_Contact> contacts = await
        _contact.GetAllByUser(HttpContext.Session.GetInt32("UserId").ToString());
        return View(contacts);
    }
    else
    {
        return RedirectToAction("Index", "Home");
    }
}

[HttpGet]
public async Task<ActionResult> Create(string id="")
{
    t_Contact contact=new t_Contact();
    if(id!="")
    {
        contact = await _contact.GetOne(id);
    }
    return View(contact);
}
```

Step 15 Controllers/ContactController.cs

```
[HttpPost]
public async Task<ActionResult> Create(t_Contact contact)
{
    if (ModelState.IsValid)
    {
        if (HttpContext.Session.GetInt32("UserId") != null)
        {
            if (contact.ContactPicture != null && contact.ContactPicture.Length > 0)
            {
                var fileName = contact.c_Email +
Path.GetExtension(contact.ContactPicture.FileName);
                var filePath = Path.Combine("../MVC/wwwroot/contact_images", fileName);
                Directory.CreateDirectory(Path.Combine("../MVC/wwwroot/contact_images"));
                contact.c_Image = fileName;
                System.IO.File.Delete(filePath);
                using (var stream = new FileStream(filePath, FileMode.Create))
                {
                    contact.ContactPicture.CopyTo(stream);
                }
            }
            contact.c_UserId = (int)HttpContext.Session.GetInt32("UserId");
            var result=0;
```

Step 15 Controllers/ContactController.cs

```
        if(contact.c_ContactId==0)
        {
            result = await _contact.Add(contact);
        }
        else
        {
            result = await _contact.Update(contact);
        }
        if (result == 0)
        {
            TempData["Message"] = "There Is Some Error while Add or Update Contact";
            return RedirectToAction("List", "Contact");
        }
        else
        {
            TempData["Message"] = "Contact Added/Updated Successfully";
            return RedirectToAction("List", "Contact");
        }
        return View();
    }
    else
    {
        return RedirectToAction("Index", "Home");
    }
}
```

Step 15 Controllers/ContactController.cs

```
    }  
    return View();  
}  
[HttpGet]  
public async Task<ActionResult> Delete(string id)  
{  
    int status = await _contact.Delete(id);  
  
    if (status == 1)  
    {  
        ViewData["Message"] = "Contact Added Successfully";  
        return RedirectToAction("List");  
    }  
    else  
    {  
        ViewData["Message"] = "There Is Some Error while Delete Contact";  
        return RedirectToAction("List");  
    }  
}  
}
```

Step 16 Steps

Create new layout

Copy `_Layout.cshtml` and `_Layout.cshtml.css` and rename with
`_ContactLayout.cshtml` and `_ContactLayout.cshtml.css`

Create folder for controller in view

`views/Contact` [avoid controller]

Create 3 files for actions for controller

`views/Contact/Create.cshtml`
`views/Contact/Index.cshtml`
`views/Contact/List.cshtml`

Step 17 View/Shared/_ContactLayout.cshtml

```
/* on top read session */
@{
    var userId = Context.Session.GetInt32("UserId");
}
...
/* Check Session */
<body>
    @if (userId==null)
    {
        <script type="text/javascript">
            window.location.href = '@Url.Action("Index", "Home")';
        </script>
    }
    <header>
...
/* Update Links */
<div class="navbar-collapse collapse d-sm-inline-flex justify-content-between"><ul class="navbar-nav flex-grow-1">
<li class="nav-item"> <a class="nav-link text-dark" asp-area="" asp-controller="Contact" asp-action="List" >List</a> </li>
<li class="nav-item"> <a class="nav-link text-dark" asp-area="" asp-controller="Contact" asp-action="Create"
asp-route-id="">Add</a> </li>
<li class="nav-item"> <a class="nav-link text-dark" asp-area="" asp-controller="Contact" asp-action="Logout" >Logout</a></li>
    </ul>
</div>
...
```

Step 18 View/Contact/Index.cshtml

```
@{
    ViewData["Title"] = "Contacts";
    Layout = "../Shared/_ContactLayout.cshtml";
}<h3>My Profile</h3>
@using Repositories
@model t_User
<br>
<br>
<div class="container row">
    <div class="col-md-3">
        
    </div>
    <div class="col-md-9">
        <div class="mt-4">
            <h3>Name : <span id="name"> @Model.c_UserName</span> </h3>
            <h4>Email : <span id="email">@Model.c_Email</span> </h4>
            <h4>Mobile : <span id="mobile">@Model.c_Mobile</span> </h4>
            <h5>Gender : <span id="gender">@Model.c_Gender</span> </h5>
            <h5>Address : <span id="address">@Model.c_Address</span> </h5>
        </div>
    </div>
</div>
```

Step 19 View/Contact/Create.cshtml

```
@{
    ViewData["Title"] = "Contacts";
    Layout = "../Shared/_ContactLayout.cshtml";
}
@using Repositories
@model t_Contact
<div class="container">
    <form enctype="multipart/form-data" asp-action="Create" asp-controller="Contact" method="post">
        <div class="modal-body">
            <input type="hidden" asp-for="c_ContactId" id="contactid" >
            <input type="hidden" asp-for="c_Image" id="image" >
            <div class="mt-3">
                <label class="control-label mt-1" for="name">Name:</label>
                <div class="mt-1">
                    <input type="text" class="form-control" asp-for="c_ContactName" id="name" placeholder="Enter name">
                </div>
            </div>
            <div class="mt-3">
                <label class="control-label mt-1" for="email">Email:</label>
                <div class="mt-1">
                    <input type="email" class="form-control" asp-for="c_Email" id="email" placeholder="Enter email">
                </div>
            </div>
        </div>
    </form>
</div>
```

Step 19 View/Contact/Create.cshtml

```
<div class="mt-3">
  <label class="control-label mt-1" for="address">Address:</label>
  <div class="mt-1">
    <input type="text" class="form-control" asp-for="c_Address" id="address"
      placeholder="Enter address">
  </div>
</div>
<div class="mt-3">
  <label class="control-label mt-1" for="mobile">Mobile:</label>
  <div class="mt-1">
    <input type="text" class="form-control" asp-for="c_Mobile" id="mobile" placeholder="Enter mobile">
  </div>
</div>
<div class="mt-3">
  <label class="control-label mt-1" for="group">Group:</label>
  <div class="mt-1">
    <input class="group-checkbox" type="checkbox" id="relative" value="Relative"
      @(Model.c_Group != null && Model.c_Group.Contains("Relative") ? "checked" : "")>
    Relative
    <input class="group-checkbox" type="checkbox" id="friend" value="Friend"
      @(Model.c_Group != null && Model.c_Group.Contains("Friend") ? "checked" : "")>
    Friend
  </div>
</div>
```

Step 19 View/Contact/Create.cshtml

```
<input class="group-checkbox" type="checkbox" id="social" value="Social"
      @(Model.c_Group != null && Model.c_Group.Contains("Social") ? "checked" : "")>
  Social
  <input class="group-checkbox" type="checkbox" id="professional" value="Professional"
    @(Model.c_Group != null && Model.c_Group.Contains("Professional") ? "checked" : "")>
    Professional
  <input type="hidden" id="c_Group" asp-for="c_Group" />
</div>
</div>
<div class="mt-3">
  <label class="control-label mt-1" for="group">Status:</label>
  <div class="mt-1">
    <select id="status" asp-for="c_Status" class="form-control">
      <option value="Favourite">Favourite</option>
      <option value="Active">Active</option>
      <option value="Inactive">Inactive</option>
    </select>
  </div>
</div>
<div class="mt-3">
  <label class="control-label mt-1" for="image">Image:</label>
```

Step 19 View/Contact/Create.cshtml

```
<div class="mt-1">
    <input type="file" asp-for="ContactPicture" class="form-control" id="Image">
    <div id="img"></div>
</div>
<div style="@string.IsNullOrEmpty(Model.c_Image) ? "display:none;" : """>

</div>
</div>
<div id="responseMessage" class="mt-3">
    <div class="alert @(TempData["Message"] != null ? "alert-danger" : "" )">
        @TempData["Message"]
    </div>
</div>
</div>
<div class="modal-footer">
    <button type="submit" class="btn btn-primary" onclick="saveData()" id="btnSave">Submit</button>
</div>
</form>
</div>
```

Step 19 View/Contact/Create.cshtml

```
<script>
  document.querySelectorAll('.group-checkbox').forEach(checkbox => {
    checkbox.addEventListener('change', () => {
      const selectedValues = Array.from(document.querySelectorAll('.group-checkbox:checked'))
        .map(cb => cb.value)
        .join(',');
      document.getElementById('c_Group').value = selectedValues;
    });
  });
</script>
```

Step 20 View/Contact/List.cshtml

```
@{
    ViewData["Title"] = "Contacts";
    Layout = "../Shared/_ContactLayout.cshtml";
}
@using Repositories
@model IEnumerable<t_Contact>
<div class="container mt-4">
    <h3>Contacts</h3>
    <div id="responseMessage" class="mt-3">
    <div class="alert @(TempData["Message"] != null ? "alert-primary" : "" )">@TempData["Message"]</div>
    </div>
    <table class="table table-striped" id="contactTable">
        <thead class="thead-dark">
            <tr>
                <th>Edit</th>
                <th width="60"></th>
                <th>Name</th>
                <th>Email</th>
                <th>Group</th>
                <th>Address</th>
                <th>Mobile</th>
                <th>Status</th>
                <th>Delete</th>
            </tr>
        </thead>
    </table>
</div>
```

Step 20 View/Contact/List.cshtml

```
</tr>
</thead>
<tbody>
  @foreach (var contact in Model)
  {
    <tr style="vertical-align: middle;">
      <td> <a href="@Url.Action("Create", "Contact", new { id = contact.c_ContactId })"
        class="btn btn-sm btn-primary">Edit</a></td>
      <td>  </td>
      <td>@contact.c_ContactName</td>
      <td>@contact.c_Email</td>
      <td>@contact.c_Group</td>
      <td>@contact.c_Address</td>
      <td>@contact.c_Mobile</td>
      <td>@contact.c_Status</td>
      <td> <a href="@Url.Action("Delete", "Contact", new { id = contact.c_ContactId })"
        class="btn btn-sm btn-danger">Delete</a></td>
    </tr>
  }
</tbody>
</table>
</div>
```